

Five classic ideas worth knowing

Trees and boosting (L09–L12) win most tabular problems today. But five **classic** methods still show up everywhere — in interviews, in libraries, and as the *ideas* behind modern tools. Each has **one beautiful, transferable idea**.

Family	Method → the idea
Learn by similarity	KNN → “you are like your neighbors”
Model how classes are generated	Naive Bayes, LDA/QDA → Bayes’ rule
Margins & kernels	SVM → widest street + the kernel trick
	Gaussian processes → kernels, now with <i>uncertainty</i>

SVM is the focus — we derive it. The other four are ideas to *recognize*, intuition-first.

Outline

Learn by similarity

Predict a point the way its nearest neighbors voted — no training, all the work at prediction time.

K-nearest neighbors: lazy learning

No training-time optimization at all. To predict a new point:

1. find its k **nearest** training points (by distance);
2. **vote** (classification) or **average** (regression).

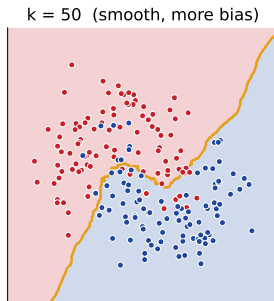
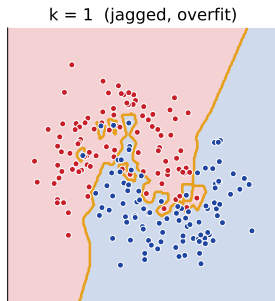
The cost is moved, not removed: *zero* training, but *expensive* prediction — every query scans the data ($O(n \cdot p)$).

k is the bias-variance knob

The boundary is **local and jagged**
(a Voronoi mosaic).

- ▶ $k = 1$: zero training error, **overfits** noise.
- ▶ large k : smoother, **more bias**.
- ▶ choose k by **CV** (callback: overfitting/CV lecture).

Predict-first: what does $k = 1$ do on noisy data? (*Memorizes it — islands around single points.*)



KNN: three gotchas

- ▶ **Scale first.** Distances are dominated by large-range features (callback: the preprocessing lecture) — standardize.
- ▶ **Curse of dimensionality.** In high dimensions everything is far from everything; “nearest” stops meaning much.
- ▶ **Inference cost.** $O(n \cdot p)$ per query $\rightarrow$ speed it up with kd-/ball-trees, or approximate nearest neighbors (ANN, FAISS) at scale.

KNN: where the idea still wins

Transferable idea — similarity search. Nearest neighbors in an **embedding space** is exactly **retrieval**: vector search, RAG, recommenders (“users like you also. . .”).

Also handy for: **KNN imputation**, **anomaly detection** (far from all neighbors), and a **quick baseline** that surprisingly often competes.

Model how each class is generated

Learn what each class *looks like*, then let Bayes' rule turn that into a decision.

Naive Bayes: from words to a decision

Is this SMS spam? We want $P(\text{spam} \mid \text{words})$. Bayes' rule:

$$\underbrace{P(y \mid \mathbf{x})}_{\text{posterior}} \propto \underbrace{P(\mathbf{x} \mid y)}_{\text{likelihood}} \underbrace{P(y)}_{\text{prior}}, \quad \text{predict } \arg \max_y P(y \mid \mathbf{x}).$$

Generative: instead of drawing a boundary, model how each class *produces* its features, then compare.

The “naive” assumption (and its variants)

Estimating the full $P(\mathbf{x} \mid y)$ is hard. **Naive** shortcut: features are **conditionally independent given the class**, so

$$P(\mathbf{x} \mid y) = \prod_{j=1}^p P(x_j \mid y).$$

Obviously false (“free” and “money” co-occur in spam) — but cheap.

Variants by feature type: **GaussianNB** (continuous), **Multinomial/BernoulliNB** (word counts / presence). **Laplace smoothing** avoids a single unseen word zeroing the whole product.

Why it works despite a false assumption

Predict-first: the independence assumption is wrong — shouldn't the classifier fail?

Why it works despite a false assumption

Predict-first: the independence assumption is wrong — shouldn't the classifier fail?

No. We only need the **argmax** to be right, not the probabilities — the estimated numbers are often badly off, but the *ranking* of classes survives.

Multiplying p simple 1-D estimates **dodges the curse of dimensionality**, needs **little data**, and is **very fast**.

By hand: message "free money", toy training counts:

	spam	ham
prior	0.4	0.6
free	0.4	0.02
money	0.3	0.05

score $\propto$ prior $\times$ likelihoods:

spam: $0.4 \cdot 0.4 \cdot 0.3 = \mathbf{0.048}$

ham: $0.6 \cdot 0.02 \cdot 0.05 = 0.0006$

$\Rightarrow$ **spam** wins $\sim 80:1$ — argmax right, even though free & money truly co-occur.

Generative vs discriminative

Generative (NB, LDA/QDA)

- ▶ models $P(\mathbf{x} \mid y)$ — “how each class looks”;
- ▶ Bayes’ rule $\rightarrow$ boundary.

Discriminative (logreg, SVM)

- ▶ models the boundary / $P(y \mid \mathbf{x})$ directly;
- ▶ no model of the features.

Naive Bayes gives a **score, not a calibrated probability** (callback: the calibration lecture) — its confident-looking numbers are usually miscalibrated.

LDA / QDA: Gaussian blobs, then Bayes

Model each class as a **Gaussian blob** in feature space; Bayes' rule turns the blobs into a boundary. This is Naive Bayes' **richer cousin** — it *allows feature correlations* (a full covariance matrix) instead of assuming independence.

Same Bayes formula as NB — **only the covariance assumption changes.**

From a blob to a score: the discriminant

Write the Gaussian for class k (mean $\boldsymbol{\mu}_k$, covariance $\boldsymbol{\Sigma}_k$, prior $\pi_k = P(y=k)$):

$$p(\mathbf{x} \mid y=k) = \frac{1}{(2\pi)^{p/2} |\boldsymbol{\Sigma}_k|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1}(\mathbf{x} - \boldsymbol{\mu}_k)\right).$$

Take log of $\pi_k p(\mathbf{x} \mid y=k)$ and drop shared constants $\rightarrow$ the **discriminant score** (predict $\arg \max_k \delta_k$):

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \log |\boldsymbol{\Sigma}_k| - \underbrace{\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k)}_{\text{(Mahalanobis distance to } \boldsymbol{\mu}_k)^2} + \log \pi_k.$$

In words: **assign $\mathbf{x}$ to the nearest class blob** — distance measured in the class's own stretched metric $\boldsymbol{\Sigma}_k^{-1}$, plus a prior nudge $\log \pi_k$.

Why shared Σ makes the boundary linear

Expand the distance term:

$$(\mathbf{x} - \boldsymbol{\mu}_k)^\top \Sigma_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k) = \underbrace{\mathbf{x}^\top \Sigma_k^{-1} \mathbf{x}}_{\text{quadratic in } \mathbf{x}} - 2\boldsymbol{\mu}_k^\top \Sigma_k^{-1} \mathbf{x} + \boldsymbol{\mu}_k^\top \Sigma_k^{-1} \boldsymbol{\mu}_k.$$

QDA (Σ_k per class): the $\mathbf{x}^\top \Sigma_k^{-1} \mathbf{x}$ term *differs* by class $\Rightarrow \delta_k$ stays **quadratic** $\Rightarrow$ the boundary is a **curve**.

LDA ($\Sigma_k = \Sigma$ shared): $\mathbf{x}^\top \Sigma^{-1} \mathbf{x}$ and $\log |\Sigma|$ are the *same* for every class $\Rightarrow$ they **cancel**.

What survives for LDA is **linear** in $\mathbf{x}$ — a hyperplane:

$$\delta_k(\mathbf{x}) = (\Sigma^{-1} \boldsymbol{\mu}_k)^\top \mathbf{x} - \frac{1}{2} \boldsymbol{\mu}_k^\top \Sigma^{-1} \boldsymbol{\mu}_k + \log \pi_k = \mathbf{w}_k^\top \mathbf{x} + b_k.$$

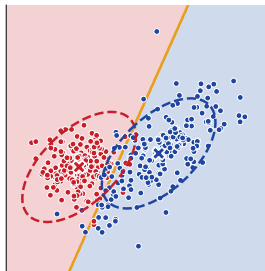
The cancellation *is* the reason: shared covariance kills the quadratic term, leaving a line.

LDA vs QDA: one knob, the covariance

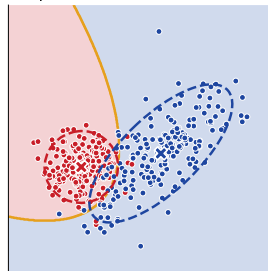
- **LDA:** *shared* covariance across classes $\Rightarrow$ a **linear** boundary (fewer parameters, more bias).
- **QDA:** *per-class* covariance $\Rightarrow$ a **quadratic**, curved boundary (more parameters, more variance).

One covariance knob: Gaussian NB (*diagonal* Σ) $\subset$ LDA (shared full Σ) $\subset$ QDA (per-class Σ_k). Fewer covariance parameters = more bias.

LDA: shared covariance $\rightarrow$ linear



QDA: per-class covariance $\rightarrow$ curved



LDA is also supervised dimensionality reduction

Beyond classifying, **LDA** finds the axes that best **separate the classes** — project onto $\leq (\text{\#classes} - 1)$ dimensions.

PCA vs LDA: PCA is *unsupervised* — it keeps the directions of most **variance**. LDA is *supervised* — it keeps the directions of most **class separation** (callback: the PCA lecture).

Predict-first: on a 2-class blob stretched diagonally, which axis would each pick?

LDA is also supervised dimensionality reduction

Beyond classifying, **LDA** finds the axes that best **separate the classes** — project onto $\leq (\text{\#classes} - 1)$ dimensions.

PCA vs LDA: PCA is *unsupervised* — it keeps the directions of most **variance**. LDA is *supervised* — it keeps the directions of most **class separation** (callback: the PCA lecture).

Predict-first: on a 2-class blob stretched diagonally, which axis would each pick?

PCA $\rightarrow$ the long (high-variance) axis; LDA $\rightarrow$ the axis that *separates the two colors*, even if it is the short one.

Strong low-data baseline; assumes roughly Gaussian features.

Margins & kernels

The centerpiece: the widest street, and a trick that bends it around any shape
— derived, not asserted.

Maximum margin: the widest street

Many lines separate these classes. SVM picks the one with the **widest margin** — the largest “safety street” around the boundary, so new points are least likely to cross.

Support vectors = the few points touching the margin edge. *Only they* determine the boundary — move any other point and nothing changes.

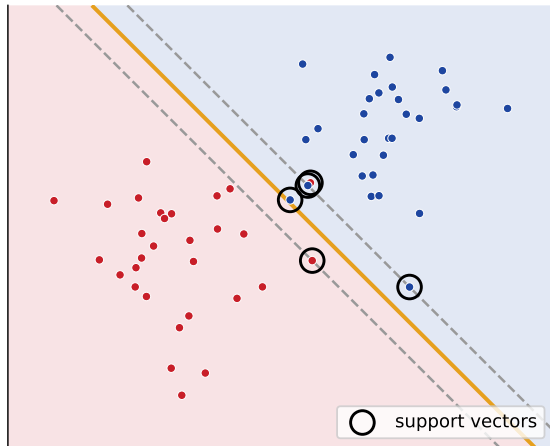
Predict-first: of several separating lines, which generalizes best?

Maximum margin: the widest street

Many lines separate these classes. SVM picks the one with the **widest margin** — the largest “safety street” around the boundary, so new points are least likely to cross.

Support vectors = the few points touching the margin edge. *Only they* determine the boundary — move any other point and nothing changes.

Predict-first: of several separating lines, which generalizes best? (*The widest-margin one.*)



From “widest street” to a quadratic program

Decision function $f(\mathbf{x}) = \boldsymbol{\theta}^\top \mathbf{x} + \theta_0$, predict $\text{sign}(f)$. The signed distance of a point is $d(f, \mathbf{x}^{(i)}) = y^{(i)} f(\mathbf{x}^{(i)}) / \|\boldsymbol{\theta}\|$.

Maximize the smallest distance γ :

$$\max_{\boldsymbol{\theta}, \theta_0} \gamma \quad \text{s.t.} \quad \frac{y^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} + \theta_0)}{\|\boldsymbol{\theta}\|} \geq \gamma.$$

Rescaling $(\boldsymbol{\theta}, \theta_0) \rightarrow (c\boldsymbol{\theta}, c\theta_0)$ leaves the *same* hyperplane, so that scale is a free choice — fix it as $\|\boldsymbol{\theta}\| = 1/\gamma$. The constraint becomes $y^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} + \theta_0) \geq 1$, and $\gamma = 1/\|\boldsymbol{\theta}\|$:

$$\begin{aligned} \min_{\boldsymbol{\theta}, \theta_0} \quad & \frac{1}{2} \|\boldsymbol{\theta}\|^2 \\ \text{s.t.} \quad & y^{(i)}(\boldsymbol{\theta}^\top \mathbf{x}^{(i)} + \theta_0) \geq 1 \quad \forall i. \end{aligned}$$

Maximizing $1/\|\boldsymbol{\theta}\| = \text{minimizing } \frac{1}{2} \|\boldsymbol{\theta}\|^2$. A **convex quadratic program** (the *primal*); margin width = $2/\|\boldsymbol{\theta}\|$.

Support vectors: sparse in the data

The points with $y^{(i)}f(\mathbf{x}^{(i)}) = 1$ sit *exactly* on the margin edge — these are the **support vectors**.

- ▶ Delete a *non*-support-vector $\rightarrow$ the solution is **unchanged**.
- ▶ Delete a support vector $\rightarrow$ the boundary **moves**.

That is why it is a *Support Vector Machine*: the model is **sparse in the data** — it depends only on a handful of points. (We will see this fall out of the dual, algebraically.)

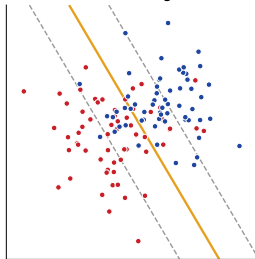
Soft margin: real data isn't cleanly separable

On non-separable data the hard-margin constraints are **contradictory** (empty feasible region). Allow **slack** $\xi_i \geq 0$ — points may sit inside/across the margin:

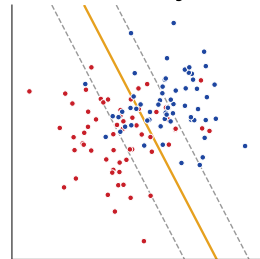
$$\min_{\theta, \theta_0, \xi} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \xi_i$$

$$\text{s.t. } y^{(i)}(\theta^\top \mathbf{x}^{(i)} + \theta_0) \geq 1 - \xi_i.$$

$C = 0.05$ (wide margin, 66 SVs)



$C = 100$ (narrow margin, 58 SVs)



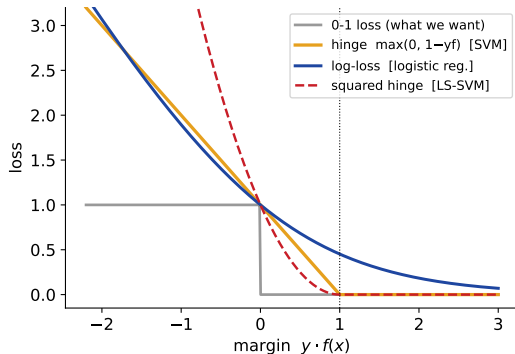
C is a regularization knob (bias-variance): large C = punish violations, narrow margin (overfit); small C = wider margin, more bias. $C \rightarrow \infty$ recovers the hard margin.

SVM is regularized ERM with the hinge loss

At the optimum, $\xi_i = \max(0, 1 - y^{(i)} f_i)$, so the soft-margin SVM is just

$$\min_{\theta, \theta_0} \frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n \max(0, 1 - y^{(i)} f_i).$$

That is **L_2 -regularized ERM** with the **hinge loss** — the same recipe as every model since L02, a new loss.



Swap the loss: hinge $\rightarrow$ **log-loss** gives *regularized logistic regression* — SVM and logreg are the **same recipe**, different loss. The hinge's *flat* part is what creates support vectors (smooth losses have none).

The dual: what the algorithm actually touches

Introduce a multiplier $\alpha_i \geq 0$ per point. Stationarity of the Lagrangian gives $\theta = \sum_{i=1}^n \alpha_i y^{(i)} \mathbf{x}^{(i)}$, and the problem becomes the **dual**:

$$\max_{\alpha} \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y^{(i)} y^{(j)} \langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle, \quad \text{s.t.} \quad \sum_{i=1}^n \alpha_i y^{(i)} = 0, \quad 0 \leq \alpha_i \leq C.$$

(1) $\alpha_i > 0$ *only* for support vectors (the sparsity, now algebraic).

(2) The data enters *only* through **inner products** $\langle \mathbf{x}^{(i)}, \mathbf{x}^{(j)} \rangle$.

Prediction, too, is only inner products: $f(\mathbf{x}) = \sum_{\text{SV}} \alpha_i y^{(i)} \langle \mathbf{x}^{(i)}, \mathbf{x} \rangle + \theta_0$ — a weighted vote of **similarities** to the support vectors. (*Remember that fact.*) (The upper bound $\alpha_i \leq C$ comes from the soft-margin slack constraints.)

Non-linear data $\rightarrow$ feature maps $\rightarrow$ a wall

Predict-first: can a straight line ever separate concentric circles?

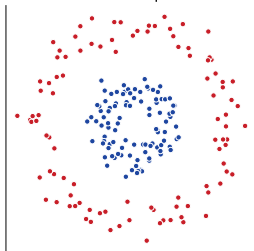
Non-linear data $\rightarrow$ feature maps $\rightarrow$ a wall

Predict-first: can a straight line ever separate concentric circles?

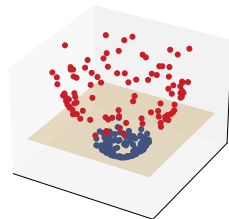
Not in 2-D. But **lift** with $\phi(\mathbf{x}) = (x_1, x_2, x_1^2 + x_2^2)$ — in 3-D a flat plane separates them (a curved boundary back in 2-D).

But explicit maps explode: degree- d monomials of p features give $\binom{d+p}{d} - 1$ terms — infeasible even for 16×16 images. Dead end?

2D: no line separates



lift $\phi = (x_1, x_2, x_1^2 + x_2^2)$: a plane separates



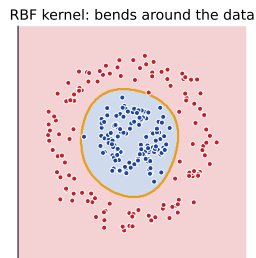
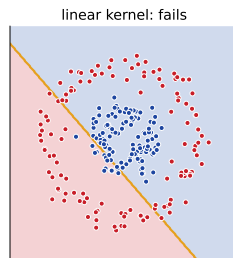
The kernel trick

From the dual, the algorithm needs *only* inner products $\langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$. A **kernel** computes that **directly**, without ever building ϕ :

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle.$$

Common kernels:

- ▶ **linear** $\mathbf{x}^\top \mathbf{x}'$;
- ▶ **polynomial** $(\mathbf{x}^\top \mathbf{x}' + b)^d$;
- ▶ **RBF** $\exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ — *implicitly infinite-dimensional*.



Replace every $\langle \cdot, \cdot \rangle$ by $k(\cdot, \cdot)$ (still convex). **RBF** = a **similarity bump** around each **support vector** — prediction is a weighted vote of similarities (the **KNN** idea again, learned).

By hand: an RBF kernel is a similarity number

Take $\gamma = 1$ and a query point $\mathbf{x} = (0, 0)$. The RBF kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ scores each support vector by **closeness**:

Near SV $\mathbf{s}_1 = (0.3, 0.1)$:

$$\|\mathbf{x} - \mathbf{s}_1\|^2 = 0.09 + 0.01 = 0.10$$

$$k = e^{-0.10} \approx \mathbf{0.90}$$

Far SV $\mathbf{s}_2 = (1, 1)$:

$$\|\mathbf{x} - \mathbf{s}_2\|^2 = 1 + 1 = 2$$

$$k = e^{-2} \approx \mathbf{0.14}$$

$k \in (0, 1]$ — a **similarity** that decays with distance. The prediction $f(\mathbf{x}) = \sum_{SV} \alpha_i y^{(i)} k(\mathbf{x}^{(i)}, \mathbf{x}) + \theta_0$ counts *near* support vectors far more than *far* ones — a soft, *learned* version of KNN.

SVM in practice — and where it still wins

```
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

clf = make_pipeline(StandardScaler(),                               # SCALE first!
                    SVC(kernel="rbf", C=10, gamma=0.1))           # tune C,
                                                                gamma by CV
clf.fit(X_train, y_train)
```

- **Scale first** (inner-product based, like KNN).
- Scales poorly: $\sim O(n^2)$ – $O(n^3)$, no big data.
- Strong on **high-dim** / **text** / **small- n** , clear margins.

Honest: on everyday tabular data, trees & boosting usually win now (L09–L12). SVM's lasting gift is the **kernel trick** — next, Gaussian processes reuse it for *uncertainty*.

Gaussian processes

The kernel idea one more time — but instead of one boundary, it returns a mean *and* calibrated uncertainty everywhere.

A distribution over functions

Don't fit *one* function — keep **all** functions consistent with the data and average them. A GP returns a **mean** prediction *and* **principled uncertainty** everywhere.

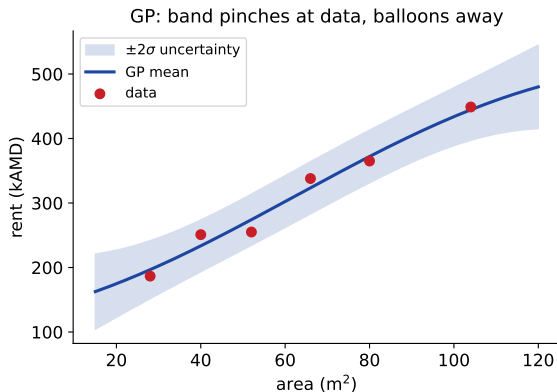
Predict-first: where is the model most uncertain?

A distribution over functions

Don't fit *one* function — keep **all** functions consistent with the data and average them. A GP returns a **mean** prediction *and* **principled uncertainty** everywhere.

Predict-first: where is the model most uncertain?

Far from the data — the band **pinches** at observations and **balloons** in the gaps.



How it works (intuition)

- ▶ The **kernel** says which points *influence* each other — similarity again, exactly the SVM kernel, **now it buys you uncertainty**.
- ▶ A Bayesian **prior over smooth functions** + the data $\rightarrow$ a **posterior** over functions.
- ▶ The kernel's **length-scale** sets how wiggly those functions may be.

Same object as the SVM kernel — a similarity function — put to a different use: propagate *how sure we are*, not just *where the boundary is*.

The one equation: condition a big Gaussian

Definition: any *finite* set of function values is **jointly Gaussian**, with covariance set by the kernel, $K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$. Observe data $\mathbf{y}$ at X ; for a test point $\mathbf{x}_*$ the joint is Gaussian, and **conditioning** gives the posterior:

$$\begin{bmatrix} \mathbf{y} \\ f_* \end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, \begin{bmatrix} K + \sigma^2 I & K_* \\ K_*^\top & K_{**} \end{bmatrix}\right) \Rightarrow$$

$$\begin{aligned} \mu_* &= K_*^\top (K + \sigma^2 I)^{-1} \mathbf{y} \\ \sigma_*^2 &= K_{**} - K_*^\top (K + \sigma^2 I)^{-1} K_* \end{aligned}$$

Mean = kernel-weighted vote of the $\mathbf{y}$'s: set $\alpha = (K + \sigma^2 I)^{-1} \mathbf{y}$, then $\mu_* = \sum_i \alpha_i k(\mathbf{x}_i, \mathbf{x}_*)$ — the **same shape** as the SVM prediction.

Variance has no $\mathbf{y}$ in it: it depends only on *where* points are. Near data K_* is large $\rightarrow$ band **pinches**; far away $K_* \rightarrow 0$ so $\sigma_*^2 \rightarrow K_{**}$ (the prior) $\rightarrow$ band **balloons**.

The “ $\Rightarrow$ ” is the standard identity for **conditioning a jointly Gaussian vector** (a linear-algebra result, quoted here — not re-derived).

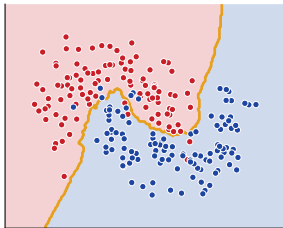
Gaussian processes: where they help

- ▶ **Principled uncertainty** — ties to the calibration thread (cf. conformal prediction).
- ▶ The engine of **Bayesian optimization** for hyperparameter tuning (callback: the tuning lecture) — propose where to try next by trading mean vs uncertainty.
- ▶ Great for **small data**, expensive experiments, scientific ML.

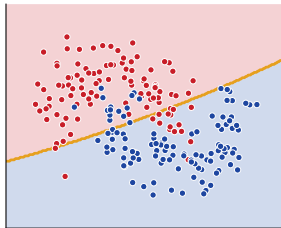
Limit: $O(n^3)$ — like kernel SVM, **not for big data**.

Same data, several ways to carve it

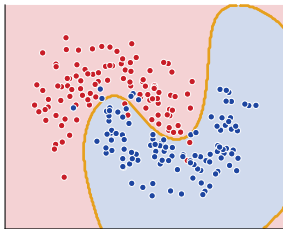
KNN (k=15): local



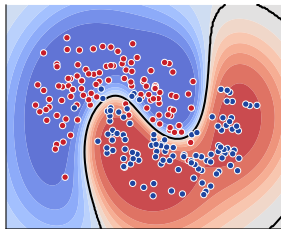
QDA: Gaussian blobs



SVM (RBF): kernel curve



SVM (RBF) proba: soft



KNN (local/jagged) · QDA (Gaussian blobs) · SVM-RBF (kernel curve) · SVM probabilities (soft) — one dataset, five ideas, different surfaces.

When would you reach for each?

Method	Scale?	Boundary	Calibrated?	Reach for it when...
KNN	yes	local/jagged	no	quick baseline; retrieval in embeddings
Naive Bayes	–	probabilistic	no	text / tiny data / very fast
LDA / QDA	–	linear / quadr.	okay	low data, $\sim$ Gaussian; LDA projects
SVM	yes	margin / kernel	no	high-dim / text / small- n , clear margin
GP	yes	smooth + band	yes	small data, need <i>uncertainty</i>

What beats them today: **trees/boosting** on tabular; **neural nets** on perception/text. These remain the *ideas* underneath.

The transferable ideas (the real point)

- ▶ **Similarity** → embeddings & retrieval (KNN).
- ▶ **Generative modeling** — “model how each class looks” (NB, LDA/QDA).
- ▶ **The kernel idea** — inner products → any similarity; the through-line of **SVM and GP**, *derived* from the dual, not asserted.
- ▶ **Bayesian uncertainty** — carry “how sure am I” everywhere (GP).

You will meet these again — in retrieval systems, in kernel methods, and in Bayesian optimization — long after you stop training a raw SVM.

Recap

- ▶ **KNN** — vote of nearest neighbors; k is the bias-variance knob; scale first.
- ▶ **Naive Bayes / LDA / QDA** — generative: model each class, apply Bayes; NB assumes independence, LDA/QDA allow correlations (linear / quadratic boundary).
- ▶ **SVM** — widest-margin street; = regularized ERM with the **hinge loss** ($\rightarrow$ logistic regression by swapping the loss); the **dual** touches data only via inner products $\Rightarrow$ the **kernel trick** bends the boundary to any shape.
- ▶ **Gaussian processes** — the kernel again, now returning **uncertainty**.

Keepers: max-margin; the dual sees only dot-products; SVM $\equiv$ logreg's cousin; the kernel trick; generative-vs-discriminative; similarity is retrieval.

HW — run the classics, compare the ideas

1. Fit `KNeighborsClassifier`, `GaussianNB`, `LinearDiscriminantAnalysis` / `QuadraticDiscriminantAnalysis`, and `SVC` (linear vs rbf) on the cheese / Titanic data; compare to your trees / logistic-regression baselines.
2. **Plot decision boundaries:** KNN $k = 1$ vs $k = 50$; linear vs RBF SVM on moons/circles. Scale the features first and show it matters for KNN & SVM.
3. Write 2–3 sentences: which idea surprised you most?

Bonus: sweep SVM C and γ and watch the boundary;
`GaussianProcessRegressor` on the 1-D rent toy (plot the uncertainty band);
`MultinomialNB` on a spam mini-set; LDA as a 2-D projection vs PCA.